

Winning the Generative Design in Minecraft Competition 2026: A Technical Report

Jort van Leenen, Robert Salden, Lana van Sprang, Niels Versteeg, and Jasper van der Zwet

LIACS, Leiden University, Leiden, The Netherlands

{j.p.van.leenen, s.m.van.sprang, n.t.versteeg, r.c.salden, j.a.van.der.zwet.2}@umail.leidenuniv.nl

1 Introduction

We aim to participate in the CONFERENCE ON GAMES 2026 [4] GDMC — AI Settlement Generation Challenge in MINECRAFT [3,7]. For this, we have developed a settlement generator that performs strongly across all evaluation criteria on unseen maps. Entries are judged in four categories:

- **Adaptability:** how well the settlement responds to terrain and environment.
- **Functionality:** how effectively it supports navigation, safety, and resource access.
- **Evocative Narrative:** whether it conveys a coherent, context-appropriate story.
- **Maintaining Aesthetics:** the consistency and quality of its visual design.

We also aim to participate in its extra *Chronicle Challenge*, which requires generating a high-quality textual artifact that is tightly coupled to and reflective of the specific settlement generated.

In general, our settlement generator can be considered to consist of a terrain analyzer, including an algorithm for splitting the building area into districts (non-buildable areas, farmland, housing districts, etc.), an A*-based algorithm that creates a network of roads, and finally, the variational generation of the buildings within those districts. The following section will detail our approach.

2 Algorithms and Approach

To generate a settlement, we follow a multi-stage pipeline in which the output of each stage serves as input for subsequent stages. The process begins with terrain segmentation, which partitions the build area into districts and classifies them according to their suitability for different purposes. Based on this classification, important landmarks such as the castle, town centre, gates, harbours, and windmills are identified. A road network is then generated to connect these locations and establish the settlement’s overall structure. Once this high-level layout has been determined, terrain modifications are performed where necessary, and buildings, farms, walls, and other structures are placed. Finally, inhabitants are generated, and a chronicle describing the settlement is created using a large language model.

Perhaps the most important component of the entire generation process is terrain segmentation. Many later decisions, including district classification, landmark placement, wall placement, and road generation, depend directly on its output. For this reason, terrain segmentation itself is implemented as a multi-phase pipeline, which is described in detail in the remainder of this section.

2.1 Phase 1 & 2: Data Acquisition and Feature Computation

In the first phase, we acquire world data that is directly exposed by MINECRAFT. Using this data, we compute and store various convenience masks and heightmaps, such as on water, terrain, and biomes, which we will use in later phases. See the top-left of Figure 1 for an illustration of the terrain heightmap with water masking applied (using hatching).

In the second phase, necessary features are computed. These include the Sobel slope, roughness, a transform based on the distance to water, the fraction of water that is locally present, the density of trees, and

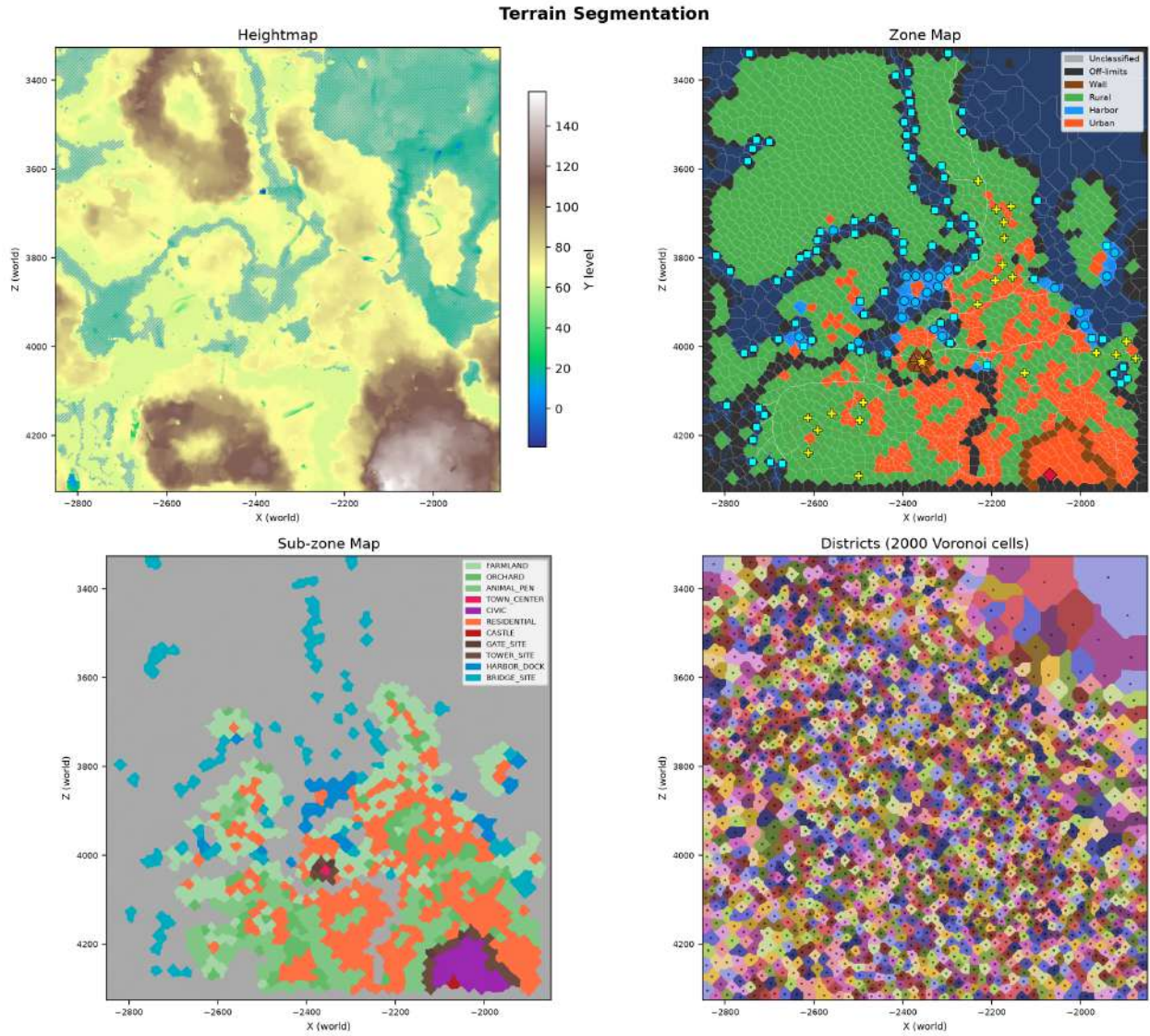


Fig. 1. The terrain map for a build area of 1000x1000 blocks. Blue stripes indicate water blocks. The image contains four subplots. Top left: the height-map (phase 1 and 2); top-right: district classifications and road network (phase 4,5 and 6); bottom-left: subzone districts (phase 4 and 5); bottom-right: Voronoi partitions (phase 3).

a mask for the border. For this, we have gotten inspiration from previous GDMC submissions. XEON0X et al.’s submission [9] uses Sobel on the heightmap with Gaussian blur before skeletonisation. We do the same but keep the raw Sobel map separate from a blurred copy because later phases need different versions. WILLIAMCWI [8] prefers flood-fill over Sobel for zone edges. We do both. Sobel gives a continuous slope gradient (better for suitability scoring), and flood-fill is used in the breadth-first search (BFS) in phase 3. Using GDPC through Python, we cannot read the world directly, which is why we cache all arrays up front in one batch call.

2.2 Phase 3: Voronoi Partitioning

In the third phase, we start partitioning the building area into (for now, unclassified) districts. See the bottom-right of Figure 1 for an illustration. This partitioning is done by creating a Voronoi partitioning [1]. The algorithm for creating this partitioning works as follows. First, some number (hyperparameter) N candidate “seed points” (located on single blocks) are scattered pseudo-randomly across the build area. Blocks that are on the border of the area, blocks that contain water, and blocks that are part of a small local area with a high slope are disregarded, and we use Poisson disk minimum spacing to avoid excessive clustering. Second, breadth-first search is initiated from these seed points. This (multi-source) BFS simply labels each block with the index of the nearest seed point, creating Voronoi cells based on the label. Then, as relaxation, each seed point is moved to the centroid of its cell (basically averaging all blocks in the cell). The BFS and this relaxation step are then iterated a fixed number of times (this is a tuned hyperparameter). Finally, an adjacency graph is created that tracks when cells, which will from now on be called districts, share an edge.

BFS is used for the Voronoi partitioning, since BFS is needed for adjacency either way, and direct Voronoi implementations that do exist, for example, in the SciPy library, do not take blocking from cliffs and water into account, which would lead to us having to correct afterwards. Poisson disk rejection is used to reduce the number of necessary iterations, as with the relaxation step, the distance would be automatically created in enough iterations. Note that we do not exclude water from this process completely; we only exclude it from being a seed point. This explains why the fraction of water in a cell is a useful metric. This will play a part in classification in the following steps; whether a cell is a harbour or determined to be a non-buildable area.

2.3 Phase 4 & 5: District Classification and Landmark Detection

After the Voronoi districts have been created, the fourth phase classifies them into functional zones. Rather than using fixed terrain thresholds, we compute continuous suitability scores inspired by previous GDMC work. In particular, we draw inspiration from NIELS-NTG’s use of terrain costs based on elevation mismatch [5] and YAWGMOTH’s buildability scoring approach [2]. Each district is evaluated using aggregated statistics that were computed in Phase 2, such as average slope, roughness, distance to water, tree density, and the fraction of water-covered cells.

Using the adjacency graph that was computed, “betweenness centrality” is computed for every district. This metric estimates how important a district is for movement through the settlement and later influences urban suitability. Urban suitability is primarily increased by flat terrain, low roughness, low water coverage, and high graph centrality, while rural suitability favours access to water and adjacency to urban districts. Harbor suitability is determined mainly by water coverage and the amount of coastline shared with non-water terrain.

Certain districts are immediately excluded from development through hard constraints. Border districts, districts with excessive water coverage, districts with steep average slopes, and districts whose dominant biome is water-related are marked as off-limits. This avoids placing structures in terrain that is either inaccessible or visually unsuitable for settlement.

The urban core is then determined using a fraction-based approach. First, a provisional castle district is identified among all buildable districts. On maps with significant elevation variation, this is the highest and most defensible district, while on flatter maps it becomes the most central district in the adjacency graph.

Urban suitability scores are biased toward this provisional castle through a distance-based bonus. Districts are then sorted by urban suitability and assigned as urban until a target fraction of approximately 22% of all buildable cells is reached. This deliberately limits city size and ensures that the majority of the map remains available for agriculture and natural terrain.

Once urban districts have been determined, neighbouring districts receive additional rural and harbour suitability bonuses. Every district is then assigned one of four zone types: `URBAN`, `RURAL`, `HARBOR`, or `OFF_LIMITS`. Small fragmented districts are merged into larger neighboring districts in an attempt to remove artifacts generated by the Voronoi process. Finally, districts of the same type are grouped into larger “super-districts” using another BFS on the adjacency graph.

The fifth phase detects specialized districts and landmark locations. First, the castle is placed in the highest urban district, with a small penalty for excessive slope. The town centre is then selected as the urban district that combines high graph centrality with proximity to the castle. Urban districts directly adjacent to the castle become civic districts intended for buildings such as the church, school, library, bank, and town hall, while all remaining urban districts are designated as residential.

The wall placement system then identifies a fortified core consisting of the castle, civic districts, and town centre. Districts immediately surrounding this core become wall districts if they provide a meaningful defensive boundary and are not already protected by water or cliffs. Gate locations are selected among wall districts that connect the city to the surrounding countryside, with preference given to lower elevations that represent natural passes. Additional wall districts that form bends or corners become tower locations.

Rural districts are further subdivided into specialised agricultural areas. Forested districts become forest buffers, while flat districts with good access to water are prioritised as farmland. Moderately sloped districts can become orchards, and remaining nearby rural districts may be assigned as animal pens. The number of agricultural districts scales with the size of the urban area, ensuring that larger settlements receive proportionally more food production.

The final landmark detection step identifies harbor docks, bridge locations, and windmill sites. Harbors are created in suitable waterfront districts adjacent to urban areas. Bridges are detected by searching for narrow water corridors that separate developed districts on opposite banks. Windmills are placed on elevated but relatively flat rural terrain close to farmland, with the number of windmills again scaling with city size.

See the top-right and bottom-left of Figure 1 for illustrations.

2.4 Phase 6: Road Network

After the settlement structure has been determined, the sixth phase generates a hierarchical road network connecting all major landmarks and districts. The road network is computed using the A* algorithm [6], with the addition that previously generated roads influence the placement of later roads through a pheromone-like reinforcement mechanism based on ant colony optimization (ACO). This should cause roads to naturally converge into shared corridors, producing a more realistic hierarchy of major roads and minor streets.

The road network is generated in three Steps. First, the primary backbone of the settlement is created. This consists of connections between the castle, town centre, gates, harbor districts, and bridge locations identified in Phase 5. Second, the residential districts are connected to the nearest existing road. Lastly, agricultural districts and windmill sites are connected to the network through smaller rural tracks.

All roads are generated using terrain-aware A* pathfinding on the block grid. The movement cost of a cell is determined by several factors. Height differences contribute a penalty proportional to elevation change, encouraging roads to follow flatter terrain. Different district types have different costs, with urban areas being preferred over rural or harbour districts. Water and off-limits districts are treated as impassable obstacles. The only exception is at designated bridge sites discovered in Phase 5, where water receives a finite traversal cost so that paths can legally cross rivers at suitable locations.

To avoid unnaturally straight roads, the cost map is randomly changed slightly after initiation. This introduces controlled asymmetry into the search process, causing roads to follow slightly curved routes that better resemble historical pathways. After each road is generated, the cells along that path receive the previously mentioned pheromone reinforcement, reducing their cost in future iterations.

2.5 Phase 7: Onwards

After the terrain separation and classification are done and the road network has been placed, we continue with placing walls around the designated wall districts, of course, depending on what the surrounding districts are (not placing walls on the edges of the wall district that connect to the city area). After this, the structure placer takes over and builds the structures based on how the districts are classified.

3 Structure and Schematic Placement

Once a district has been classified and the road network established, buildings are placed using a unified schematic-based placement system. Each structure corresponds to a unique MINECRAFT build that was adapted where necessary, saved as a schematic, and exported to a CSV representation. Our library contains roughly 100 hand-built (not all original) schematics spanning twelve categories, including about thirty house variants alongside civic buildings, market stalls, fountains, and other decorative structures. The CSV records every non-air block together with its local position, block ID, block states (e.g., orientation or waterlogged status), and any associated tile-entity data. During initialization, each CSV is parsed into a **Building** descriptor containing its dimensions, footprint, shadow volume, entrance information, and connector metadata.

The footprint represents all occupied cells at ground level and is used for terrain evaluation and collision checks. The shadow volume consists of all occupied cells at any height and prevents collisions involving overhanging elements such as roofs. Entrance locations are derived from the main door and from special *jigsaw* marker blocks embedded within the schematic. These markers encode entrance and approach connectors used during path generation. A building’s category (e.g., house, church, windmill, barn) is inferred from its filename and used to enforce category-specific quotas and placement preferences. For example, only a single town hall may be placed, while profession-related buildings, such as the butcher, fletcher, and cartographer, are preferentially assigned to residential subzones. This ensures that settlements contain a balanced and varied mix of structures.

For each placement, candidate anchor positions within the target district, or at pre-selected landmark locations such as the castle or windmill sites, are evaluated across all four possible rotations. Candidates are scored according to several criteria: proximity to the road network, terrain suitability, orientation, and accessibility. Terrain suitability is determined from the height variation and slope across the footprint, with excessively uneven terrain rejected outright rather than aggressively flattened. Certain civic buildings receive an orientation bonus when facing the settlement centre. A boundary-alignment term rewards configurations whose entrances or jigsaw connectors face an open, road-adjacent side of the footprint rather than being obstructed by neighbouring structures.

The highest-scoring candidate is selected for placement. Vegetation within the footprint is cleared, the terrain is levelled relative to the entrance elevation, and foundations and supporting walls are generated beneath the structure where necessary. A terraced apron is then constructed around the perimeter to blend the building smoothly into the surrounding terrain.

The structure itself is placed block-by-block using GDPC’s transformation system, which applies the selected rotation and translates the schematic to its final world position. Jigsaw marker blocks are not placed in the world; instead, they serve as attachment points from which short approach paths are generated toward the nearest road using A* pathfinding. Path elevations are interpolated between the entrance and the road surface, after which the path is covered with mud blocks.

Before placement, blocks pass through a biome-aware palette transformation that substitutes wood, stone, and foundation materials with biome-appropriate variants. Additional stochastic substitutions introduce minor visual variation between otherwise identical structures. Finally, command blocks embedded in the schematic are replaced by a dedicated furniture placement system that inserts functional interior elements such as chests, beds, and crafting tables. To reduce overhead, parsed schematic descriptors are cached and reused whenever the same building type is placed multiple times, avoiding repeated CSV parsing during settlement generation.

A schematic’s command-block markers only specify *where* interior objects may go, not *what* they are. During furniture placement, neighbouring markers are grouped into stacks and clusters, and each stack is

independently assigned a furniture piece from either a storage/work-surface pool (chests, barrels, bookshelves, furnaces) or a single-block utility pool (looms, lecterns, crafting tables, anvils, and similar), occasionally combined into a shared pile or hung from the ceiling on chains, and finished with a small decoration such as a candle or potted plant. The interior is therefore not fixed for a given building type: the same schematic produces a different furnishing arrangement on every placement, while the marker layout keeps the room composition recognisable. Container contents are randomised separately from a loot table keyed to the building’s category (e.g., house, church, farm) and, for houses, its size, so a blacksmith’s chest and a farmer’s barrel are stocked differently. Figure 2 shows an example interior.



Fig. 2. An example interior generated from furniture markers embedded in a house schematic.

4 Chronicle Generation

After the settlement has been fully generated, a chronicle is created to provide narrative context for the world. Unlike the previous stages, the chronicle generation step operates on the completed settlement and produces a written document that is placed in-game on a lectern.

The chronicle system gathers information about the generated settlement, including its surrounding biomes, building composition, presence of a harbour, marketplace, castle, and any generated DnD-like disasters. This information is used to create a structured description of the settlement.

To improve specificity, visual information is also provided to the language model. This includes the district map generated during terrain segmentation and a regional overview obtained from the MINECRAFT world seed and settlement location. These inputs allow the model to incorporate details about both the settlement layout and its surroundings.

Using these textual and visual inputs, a prompt is constructed and submitted to GPT-5. The model is instructed to generate a medieval-style document that is specific to the generated settlement. Finally, the generated text is split into pages compatible with MINECRAFT’s written book format and placed on a lectern within the settlement.

5 Tools and Libraries

Our settlement generator is implemented in Python and uses the GDMC Python framework (GDPC) for interaction with MINECRAFT.¹ Additional libraries are used throughout the pipeline: NumPy, SciPy, and scikit-image for terrain analysis; Pandas for data modeling and manipulation; Matplotlib for visualization;

¹ <https://github.com/avdstaaij/gdpc>

nbtlib for parsing MINECRAFT’s NBT data format; and the OpenAI Python library for generating a narrative chronicle of the settlement using an LLM.

We also contributed a minor improvement to GDPC by addressing a networking-related issue that we encountered during the Settlement Generation Challenge and that many students had previously encountered while working on the first assignment of the course.² We also intend to share our schematic-parsing and jigsaw/furniture-marker tooling with the community, as it is largely independent of the rest of our pipeline and could benefit other GDMC entries.

6 Experimentation

To assess the procedural generation process, we generate villages of three building area sizes: 1000×1000 , 512×512 , and 256×256 blocks, representing the real competition build areas, and look at the (subjective) quality of the settlements.

During development, the generator was instead validated progressively: first on a flat world, where the terrain could not mask placement errors, then on plain terrain, and finally on increasingly rugged maps chosen to expose edge cases. This continued until we could no longer identify obvious errors or ran out of time before the submission deadline, whichever came first.

6.1 Large terrain (1000×1000)

The villages that are generated well are shown in Figures 3, 4, 5, 6, 7.



Fig. 3. The generated village with a building area of 1000×1000 blocks in the mesa biome, showing one of the roads and a building.

² <https://github.com/avdstaaij/gdpc/pull/121>



Fig. 4. The generated village with a building area of 1000x1000 blocks in a desert and mesa biome. The image shows the overview from above.



Fig. 5. The generated village with a building area of 1000x1000 blocks in a mesa biome. The image shows the generated castle.



Fig. 6. The generated village with a building area of 1000x1000 blocks. The image shows the generated windmills.



Fig. 7. The generated village with a building area of 1000x1000 blocks. The image shows the generated docks.

Bad instances The villages that were not generated well or had some mishaps are shown in Figures 8, 9, and 10. Throughout these generations, we noticed that some builds were placed frequently, despite the district already containing multiple of the same structures. For example, in some generations, there were too many windmills in an area, which is unrealistic for one village. In Figure 9, there are too many piers extending from the dock body.



Fig. 8. The generated village with a building area of 1000x1000 blocks. The image shows a bad generation of the windmill in the mesa biome.

6.2 Medium terrain (512×512)

The generated villages with a medium terrain of 512×512 are shown in Figures 11, 12, 13 and 14.

Bad instances As shown in Figures 15 and 16, some poor generations occurred in the medium terrain setting. However, these were less frequent than in the large terrain setting.

6.3 Small terrain (256×256)

Compared to the generations of the large and medium terrains, the small terrain had few bad instances.

6.4 Discussion

As seen from the results, the algorithm struggled with large terrains. It placed some structures too frequently in an area, resulting in an unrealistic village and thus a bad or ‘ugly’ generation, such as the amount of piers in Figure 9. This is likely due to the large building area. The algorithm likely assessed that there was enough space in the district to fill and placed an extra structure. This may be the reason why there are many



Fig. 9. The generated village with a building area of 1000x1000 blocks. The image shows a bad generation of the docks along the village: there are too many docks in a small space.



Fig. 10. The generated village with a building area of 1000x1000 blocks. The image shows a bad generation of the market square.



Fig. 11. The generated village with a building area of 512x512 blocks. The image shows an overview from above of the village.



Fig. 12. The generated village with a building area of 512x512 blocks. The image shows a generated village house and the neighbouring houses.



Fig. 13. The generated village with a building area of 512x512 blocks. The image shows the generation of a bridge across the river.



Fig. 14. The generated village with a building area of 512x512 blocks. The image shows the generation of houses on a hillside.



Fig. 15. The generated village with a building area of 512x512 blocks. The image shows a bad generation of the market square.



Fig. 16. The generated village with a building area of 512x512 blocks. The image shows a bad generation of a village house.



Fig. 17. Generated village in the mountains and tundra with a 256x256 building area. The image shows an overview of the village from above.



Fig. 18. Generated village in the mountains and tundra with a 256x256 building area. The image shows a sideways overview of the village.



Fig. 19. Generated village with a 256x256 building area with an in-depth view of one of the roads and the village wall.



Fig. 20. Generated village with a 256x256 building area with an in-depth view of one of the farm lands, showing a wheat field.

duplicate structures in certain areas, resulting in bad generations. This is consistent with the results we found for the medium and small terrain, where the medium terrain had fewer bad instances than the large terrain, and the small terrain had little to none. For the small area size, the algorithm likely decided that there was little space left after placing structures and stopped placing them at the right amount, which resulted in a good generation. However, if the building size is too small, such as below the 256×256 area size, not all structures are placed, leading to a bad generation.

We also noticed that the algorithm struggled with adaptability in harsh terrains, such as cliffs and mountains. Structures such as the market square require flat surfaces, and to account for this, the area around the market square is flattened and sometimes placed higher than the original surface. However, as seen in Figure 15, this resulted in market squares that are placed too high and don't match the rest of the village. The bad adaptability in harsh terrains is also likely the reason that some houses are not placed well, such as in Figure 16. Moreover, the removal of the original vegetation in the area didn't work at all times, meaning that this is also an area of improvement.

However, the structures are well adapted in environments that are relatively flat. As seen in Figure 13, there is a bridge built across the river to access the other area and to access the docks. Due to the analysis of the terrain, the structures are often well-adapted to the environment. Additionally, the block pallet of the structures is different depending on which biome the village locates: snowy biomes swap wooden elements for stone, giving buildings a heavier appearance, while jungle and desert villages shift towards mossy stone and sandstone, respectively.

Regarding aesthetics, the structures are all well aligned and fit the medieval town aesthetic. The generation of the different areas and structures is also mostly consistent. The medieval castle and the walls are always generated, and the buildings and the roads to structures as well. There is even a difference between the main roads and the side roads due to the mapped subzones. The structures and the zones are also believable and coherent. For example, the orchards and farmland are always placed outside the centre of the village, and the docks are built when there is a river or an ocean nearby. Moreover, the village contains villagers, animals, and sometimes has a deadly disaster, such as a dragon, for believability and aesthetics. And, regarding the chronicle challenge, the chronicle text is fitting and varying in each generation due to the dynamic prompting. Grounding the model in screenshots and structured game-state information kept it from inventing details, though it also confined the chronicles to what that information could support, making them somewhat generic; we consider this the least polished part of our submission.

Lastly, the structures are functional. The generated roads support the navigation of the player, and there are nearby buildings with a functional interior design, such as chests, beds, and lanterns. The village has farmlands and orchards in case the player has to harvest crops and wood.

7 Use of Generative AI

For software development, we primarily used Claude Code through a one-month Claude Pro subscription. We found it particularly effective for larger software engineering tasks that required reasoning across multiple files and maintaining awareness of a substantial codebase and project context. This made it well suited to the overall architecture, integration, and iterative development of the settlement generator. For smaller and more localized changes, we also made use of freely available models, including ChatGPT, Claude, and Gemini. In our experience, Claude generally performed best on software engineering tasks related to this project.

Generative AI was used as a development assistant rather than as an autonomous developer. When a desired implementation was already clear, we provided detailed instructions and used the model to accelerate coding tasks. All generated code was reviewed and validated before being incorporated into the project. In cases where we were uncertain about the best approach to a problem, we used the models for brainstorming, design discussions, and literature or web-based research.

One example is the terrain segmentation system. While we independently arrived at the idea of using Voronoi-based district generation, we initially encountered difficulties adapting the approach to the wide variety of terrain found in MINECRAFT worlds. We then used an LLM to research potential improvements and alternative approaches. After reviewing and evaluating the resulting suggestions, we selected the ideas we

considered most promising and guided their integration into the final implementation. The resulting system was therefore not generated automatically, but developed through an iterative process in which AI-assisted research informed our own design decisions.

Generative AI was also used during the preparation of this report, primarily functioning as a writing assistant. Namely, the report content, structure, technical descriptions, and conclusions were written by us, and AI tools were used to improve local phrasing, sentence structure, readability, and conciseness, similar to the role of a proofreader. All final text was reviewed and edited by us to ensure technical correctness and accurate reflection of the work that was performed.

8 Conclusion and Future Work

Our submission placed first among ten entries in the ninth GDMC Settlement Generation Challenge and received the highest *Maintaining Aesthetics* score recorded in the competition’s history, the category we are most proud of. We are very satisfied with the final result. MINECRAFT worlds are highly dynamic and diverse, yet our system is capable of generating convincing and realistic settlements across a wide range of environments, assuming a reasonable build area selection. The resulting settlements exhibit a strong degree of variety while maintaining a believable and coherent overall structure.

Nevertheless, there always remains room for improvement. One direction for future work is to further extend terrain segmentation and environmental analysis. This would enable more sophisticated settlement layouts and increase both realism and diversity. For example, road generation could distinguish between major roads and smaller alleyways, allowing different placement and decoration strategies. Similarly, additional structure types could be introduced, such as landmarks and specialized districts. Examples include statues, observatories, industrial zones, warehouses near harbors, or other unique buildings that help reinforce the identity of a settlement.

Another promising direction is the refinement of the existing system. Settlement generation involves many interacting components, each of which influences the quality of the final result. While significant effort was invested in designing these components and ensuring they work together effectively, the complexity and unpredictability of MINECRAFT’s terrain generation occasionally still expose limitations in the current approach. Further tuning and robustness improvements could therefore lead to more consistent results and allow the system to better handle challenging terrain configurations.

References

1. Aurenhammer, F.: Voronoi diagrams—a survey of a fundamental geometric data structure. *ACM Computing Surveys* **23**(3), 345–405 (Sep 1991). <https://doi.org/10.1145/116873.116880>
2. Eger, M.: yawgmoth/GDMC25 (Jun 2025), <https://github.com/yawgmoth/GDMC25>, original-date: 2025-06-15T00:26:13Z
3. IEEE: 2026 Settlement Generation Competition - Generative Design in Minecraft, <https://gendesignmc.wikidot.com/wiki:2026-settlement-generation-competition>
4. IEEE: CoG 2026, <https://cog2026.org/?seccion=areaCientifica&subSeccion=contenido&id=8#gdmc>
5. Poldervaart, N.: Niels NTG Poldervaart / GDMC - Generative Design in Minecraft, <https://nielspoldervaart.nl/gdmc>
6. Russell, S., Norvig, P.: *Artificial Intelligence: A Modern Approach* (4th Edition). Pearson (2020)
7. Salge, C., Green, M.C., Canaan, R., Togelius, J.: Generative design in minecraft (GDMC): settlement generation competition. In: Dahlskog, S., Deterding, S., Font, J.M., Khandaker, M., Olsson, C.M., Risi, S., Salge, C. (eds.) *Proceedings of the 13th International Conference on the Foundations of Digital Games, FDG 2018, Malmö, Sweden, August 07-10, 2018*. pp. 49:1–49:10. ACM (2018). <https://doi.org/10.1145/3235765.3235814>
8. William: williamcwi/GDMC (Nov 2025), <https://github.com/williamcwi/GDMC>, original-date: 2021-04-08T16:27:05Z
9. Xeon0X: Xeon0X/GDMC-2024 (Feb 2026), <https://github.com/Xeon0X/GDMC-2024>, original-date: 2024-04-19T21:17:10Z